

Bloque 04_linux

Índice de módulos

- Linux capabilities
- Tareas programadas y fronteras de confianza
- Docker y escapes de contenedor
- Enumeración razonada para escalada Linux
- Grupos peligrosos (docker, lxd, disk)
- Exploits de kernel (ultimo recurso)
- NFS con no_root_squash
- Secuestro de PATH
- Python library hijacking
- Análisis y abuso de reglas sudo
- Sudo con env_keep (LD_PRELOAD)
- Binarios SUID y credenciales efectivas
- Inyeccion por comodin (wildcard)
- Ficheros sensibles escribibles

Linux capabilities

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Las capabilities parten los privilegios de root en piezas concretas que se asignan a un binario sin darle SUID completo. Es una version mas granular del suid: en vez de 'corre como root', es 'puede hacer esta cosa de root'. Algunas capabilities permiten escalar directamente.

Se usan para que un binario haga una tarea privilegiada (abrir un puerto bajo, cambiar de uid) sin ser root del todo. Pero cap_setuid permite a un interprete (python, perl) cambiar su uid a 0, y cap_dac_read_search permite leer cualquier fichero. Mal puestas, son root.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeracion (enum-privesc-linux), justo despues de SUID. getcap recorre el sistema buscandolas.

Cómo identificarla

- getcap lista un binario con cap_setuid+ep (escalada directa via python/perl).
- cap_dac_read_search+ep: lectura de /etc/shadow y cualquier fichero protegido.
- cap_setuid sobre un interprete no estandar puesto a proposito en el reto.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

getcap -r / 2>/dev/null y busca cap_setuid o cap_dac_read_search en la salida.

Evidencia esperada: Una linea 'binario = cap_setuid+ep' sobre un interprete (python/perl); tras el abuso, euid=0.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
getcap -r / 2>/dev/null
```

Objetivo y contexto: Lista todos los binarios con capabilities asignadas. -r recorre recursivamente; el ruido de permiso denegado se oculta con 2>/dev/null.

Resultado esperado: Líneas 'binario = cap_xxx+ep'. cap_setuid -> escalada directa (GTF0Bins).

cap_dac_read_search -> lees /etc/shadow. Ignora los del sistema base.

Capa 2: prueba documentada

```
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Objetivo y contexto: Si python tiene cap_setuid, setuid(0) cambia tu uid a root y luego lanzas una shell que ya corre como root. Es la receta de GTF0Bins para esa capability.

Resultado esperado: Prompt de root y id con uid=0. Cambia python por el interprete concreto que getcap marco con cap_setuid.

Anatomía de los payloads

La primera prueba es `getcap -r / 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una línea 'binario = cap_setuid+ep' sobre un interprete (python/perl); tras el abuso, `euid=0`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Recorre el sistema con `getcap -r` buscando capabilities en binarios.
- Identifica la capability y el binario en GTF0Bins (sección Capabilities).
- Si es cap_setuid en python/perl, fuerza `setuid(0)` y lanza una shell.
- Confirma `uid=0` y recoge la flag.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `getcap -r /` no devuelve ningún binario fuera de los estándares del sistema.
- Las capabilities presentes no incluyen cap_setuid ni cap_dac_read_search (son inofensivas para `privesc`).

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Permisos finos que dan a un binario poderes de root concretos; cap_setuid sobre un interprete es escalada directa. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(capabilities\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Tareas programadas y fronteras de confianza

Objetivos de aprendizaje

Reconstruir identidad, calendario, shell, entorno y working directory de una tarea; demostrar consumo de un recurso escribible con marcador inocuo.

Prerrequisitos

Procesos, permisos, shell, PATH, cron y observación temporal.

Fundamentos técnicos

Cron ejecuta según un entorno propio. En crontabs habituales, SHELL suele ser /bin/sh; HOME y LOGNAME derivan del propietario, y PATH puede ser limitado o explícito. Un script escribible solo es vector si la tarea privilegiada lo ejecuta realmente.

Modelo mental

schedule -> propietario -> entorno/cwd/shell -> comando -> recurso controlable
-> ejecución -> efecto observable

Superficie de ataque

Crontabs de sistema/usuario, /etc/cron.*, scripts, configs, wildcards, rutas relativas y servicios/timers equivalentes.

Cómo identificarla

- pspy correlaciona tiempo, UID y comando.
- Permisos efectivos permiten modificar el recurso consumido.
- Un marcador aparece tras el ciclo con propietario esperado.
- Restaurar el recurso elimina el efecto.

Preguntas que debo hacerme

1. ¿Qué usuario ejecuta?
2. ¿Cuándo y con qué shell?
3. ¿Qué PATH, HOME y cwd usa?
4. ¿Qué archivo/directorio controla?
5. ¿Cómo pruebo consumo sin shell persistente?

Prueba mínima

Agregar en un laboratorio un marcador reversible que escriba fecha y id a /tmp, conservar copia y hash del original, observar un ciclo y restaurar.

Construcción progresiva del payload

1. Observar dos ciclos.
2. Capturar UID/comando.
3. Verificar permisos del recurso y directorios.
4. Reproducir con entorno mínimo.
5. Añadir marcador.
6. Confirmar y limpiar.

Anatomía de los payloads

- Contexto de entrada: root ejecuta `/opt/jobs/report.sh` cada minuto.
- Sintaxis original: script POSIX sh.
- Entrada controlada: contenido escribible por grupo.
- Transformaciones conocidas: cron aplica su entorno y entrega el script a `/bin/sh`.
- Parser final: `/bin/sh` de cron.
- Sink: creación de fichero.
- Primitiva: escribir marcador root en `/tmp/cron-proof`.
- Payload mínimo: una línea de `id` redirigida al marcador.
- Significado de cada componente: `id` revela la identidad efectiva y la redirección conserva evidencia en un destino inocuo.
- Resultado esperado: marcador creado como root en el siguiente ciclo.
- Control negativo: hash original y ciclo sin cambio.
- Restricción observada: PATH mínimo.
- Por qué falla la variante básica: comando auxiliar sin ruta no se encuentra.
- Hipótesis de adaptación: usar ruta absoluta o definir entorno explícito.
- Payload adaptado: ruta absoluta de la utilidad permitida.
- Por qué debería funcionar: elimina la búsqueda dependiente del PATH de cron.
- Evidencia: archivo root tras el ciclo.
- Cuándo no funcionaría: tarea no activa, otro usuario, script no consumido o MAC.

Variaciones según el contexto

Cron de usuario, `/etc/crontab` y directorios periódicos expresan usuario de forma distinta. Systemd timers no heredan necesariamente las mismas variables ni usan shell.

Filtros y bypasses

Un fallo por PATH no se arregla cambiando payload sin medir entorno. Reproduce `env -i` con variables observadas y rutas absolutas.

Evidencias de confirmación

Marcador consumido por la tarea y creado con identidad superior. Permiso `w` aislado no basta.

Escalado de impacto

Evaluar capacidad de escritura/ejecución y restaurar siempre. Evitar reverse shells cuando un fichero marcador demuestra la relación.

Errores frecuentes

- Asumir entorno interactivo.
- Esperar un ciclo sin confirmar calendario.
- Modificar sin backup.
- Confundir proceso periódico de otro usuario con root.
- Mezclar wildcard, PATH y script escribible.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
manual funciona, cron no	entorno/cwd/shell	capturar env, usar rutas absolutas
nunca aparece proceso	tarea inactiva/calendario	observar varios ciclos y logs
marcador es del alumno	ejecución manual propia	timestamp y PID/UID de pspy
script cambia pero efecto no	copia distinta o caché	ruta de /proc/PID/cmdline, inode/hash
command not found	PATH mínimo	ruta absoluta y PATH registrado

Mitigaciones

Scripts root:root no escribibles, directorios seguros, rutas absolutas, entorno explícito, privilegio mínimo, locking/logging y revisión de tareas obsoletas.

Relación con pentesting y certificaciones

Se evalúa reconstrucción del contexto de ejecución y limpieza.

Caso guiado

Caso A — Básico: script escribible consumido

pspy confirma UID 0 y el script de grupo. Un marcador reversible aparece como root tras un ciclo. Conclusión: ejecución privilegiada confirmada.

Caso de adaptación

Caso B — Funciona en terminal, falla en cron

El script llama custom-tool sin ruta. Experimento: capturar PATH de cron. Resultado: no contiene /opt/tools. Adaptación: ruta absoluta. Conclusión: el fallo era entorno, no permisos.

Caso C — Transferencia: timer de mantenimiento

Un timer systemd ejecuta un binario que lee config escribible. El alumno usa marcador de ruta y observa UID. Misma relación, distinto scheduler y sin asumir /bin/sh.

Caso D — Falso positivo: copia abandonada

/opt/jobs/report.sh.bak es escribible, pero la tarea ejecuta report.sh con inode distinto. Conclusión: archivo interesante no consumido.

Ejercicios

1. Reconstruye entorno mínimo de una línea crontab.
2. Diseña una prueba de consumo reversible.
3. Distingue cron de systemd timer.
4. Explica por qué un backup escribible puede ser falso positivo.

Resumen

Cron aporta tiempo e identidad; el vector aparece cuando consume una superficie controlada bajo ese contexto.

Chuleta operativa

1. Dos ciclos.

2. UID y comando.
3. Shell/PATH/HOME/cwd.
4. Permisos e inode.
5. Marcador.
6. Restauración.

Referencias

- [crontab\(5\)](#)
- [Concepto interno](#) (cron-abuse)

Navegación

Anterior: [SUID](#). Siguiente: [PATH hijacking](#). Índice: [curso](#).

Docker y escapes de contenedor

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un contenedor comparte kernel con el host. El acceso al socket Docker, un contenedor privilegiado o un bind mount del filesystem del host puede convertir una shell en el contenedor en control del host.

El daemon Docker actúa con privilegios del host y los montajes trasladan esos permisos al proceso dentro del contenedor.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Tras obtener shell en un contenedor o cuando id/groups muestra docker y el socket /var/run/docker.sock es accesible.

Cómo identificarla

- docker.sock
- --privileged
- CAP_SYS_ADMIN
- mount /host
- grupo docker

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Identifica socket, capacidades, mounts y privilegios; confirma impacto solo con un fichero de prueba no destructivo.

Evidencia esperada: Sabes si el contenedor es una frontera real o una vía de escritura/lectura sobre el host.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
id; ls -l /var/run/docker.sock; capsh --print 2>/dev/null
```

Objetivo y contexto: Recoge grupo, socket y capacidades antes de intentar cualquier acción.

Resultado esperado: Permisos del socket y capabilities efectivas.

Capa 2: prueba documentada

```
docker ps --format '{{.ID}} {{.Image}} {{.Mounts}}'
```

Objetivo y contexto: Comprueba si puedes hablar con el daemon y que montajes existen.

Resultado esperado: Contenedores y montajes visibles al usuario actual.

Anatomía de los payloads

La primera prueba es `id; ls -l /var/run/docker.sock; capsh --print 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Sabes si el contenedor es una frontera real o una vía de escritura/lectura sobre el host.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica runtime y usuario.
- Revisa socket, capabilities y mounts.
- Prueba acceso a un recurso de laboratorio.
- Registra la remediación: retirar socket, privilegios y mounts innecesarios.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El proceso está en una VM sin Docker accesible.
- El socket no existe y no hay privilegios especiales ni montajes del host.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Distinguir aislamiento normal de montajes, sockets y privilegios que permiten alcanzar el host. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (docker-container-escape)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeración razonada para escalada Linux

Objetivos de aprendizaje

Convertir salidas de enumeración en relaciones verificables de permisos, priorizar por precondiciones y descartar falsos positivos antes de ejecutar una técnica.

Prerrequisitos

UID real/efectivo, grupos, permisos, procesos, filesystem, entorno y alcance de laboratorio.

Fundamentos técnicos

Un hallazgo no es un vector hasta completar esta cadena:

objeto/proceso privilegiado -> operación que realiza -> dato que controlo
-> momento/condición de consumo -> identidad efectiva -> capacidad obtenida

El color de una herramienta indica prioridad heurística, no explotación ni impacto.

Modelo mental

Para cada pista anota consumidor privilegiado, superficie controlable, frontera rota, precondiciones, prueba inocua y control negativo.

Superficie de ataque

Reglas sudo, ejecutables SUID/capabilities, tareas, servicios, grupos, credenciales, ficheros sensibles, rutas/bibliotecas y kernel. El orden depende de evidencia y coste, no de una lista rígida.

Cómo identificarla

- Un proceso de UID superior consume un archivo, nombre, ruta u opción controlable.
- La precondición se demuestra con permisos efectivos, no solo ls aislado.
- Un marcador inocuo aparece con identidad esperada.
- El control sin influencia no produce el efecto.

Preguntas que debo hacerme

1. ¿Qué identidad obtendría realmente?
2. ¿Qué controlo: contenido, nombre, directorio, argumento o entorno?
3. ¿Quién lo consume y cuándo?
4. ¿Qué defensa anula la relación?
5. ¿Cuál es la capacidad mínima demostrable?

Prueba mínima

id, grupos, sudo -l, montajes y procesos aportan contexto. Para una pista concreta, usa un marcador reversible que pruebe consumo e identidad sin crear shell ni modificar datos sensibles.

Construcción progresiva del payload

1. Registrar identidad actual.
2. Formular relación concreta.
3. Verificar cada precondición por separado.
4. Observar consumidor privilegiado.
5. Introducir marcador inocuo.

6. Confirmar identidad y limpiar.

Anatomía de los payloads

- Contexto de entrada: pspy muestra `/usr/local/bin/backup` con UID 0.
- Sintaxis original: el script lee `/opt/team/job.conf`.
- Entrada controlada: fichero de configuración escribible por grupo.
- Transformaciones conocidas: parser key/value.
- Parser final: script de backup.
- Sink: creación de archivo como root.
- Primitiva: cambiar solo la ruta de salida a un marcador permitido.
- Payload mínimo: valor `/tmp/backup-proof`.
- Significado de cada componente: `/tmp` satisface la raíz permitida; `backup-proof` identifica la prueba.
- Resultado esperado: el consumidor privilegiado crea el marcador con UID 0.
- Control negativo: configuración sin cambio.
- Restricción observada: solo rutas bajo `/tmp` aceptadas.
- Por qué falla la variante básica: una ruta fuera de `/tmp` es rechazada por el parser.
- Hipótesis de adaptación: medir la capacidad mínima dentro de la raíz admitida.
- Payload adaptado: `/tmp/backup-proof` con token único.
- Por qué debería funcionar: conserva la gramática aceptada y cambia solo el destino observable.
- Evidencia: archivo UID 0 con contenido esperado.
- Cuándo no funcionaría: config releída por otro usuario, firma, permisos o ruta fija.

Variaciones según el contexto

Automatización encuentra amplitud; comprobaciones manuales explican causalidad. Versiones, mounts, contenedores, MAC y `no_new_privs` pueden cambiar la validez de una pista.

Filtros y bypasses

No existe “bypass” general. Si falla una precondition, descarta o busca otra relación. No fuerces una receta porque un nombre aparezca en una base de datos.

Evidencias de confirmación

Consumo controlado por una identidad superior con efecto mínimo atribuible. NOPASSWD, un bit SUID o un directorio escribible son hallazgos, no confirmación de escalada.

Escalado de impacto

Después del marcador, evaluar la capacidad exacta: lectura, escritura, ejecución o cambio de identidad. Ampliar solo si el objetivo del laboratorio lo exige.

Errores frecuentes

- Priorizar colores sin consumidor.
- Ejecutar herramientas agresivas antes de contexto básico.
- Confundir propietario con UID efectivo del consumidor.
- Ignorar mount, entorno y versión.
- Saltar al payload final.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
fichero escribible, nunca leído	falso positivo	observar accesos/mtime/proceso
proceso periódico no es root	impacto lateral, no root	UID real del proceso
SUID no cambia EUID	nosuid, no_new_privs o binario baja privilegio	mount, /proc, id -ru/-u
sudo regla no coincide	argumentos/ruta/host	leer salida completa de sudo -l
herramienta marca kernel	versión parcheada/backport	paquete y advisory del proveedor

Mitigaciones

Privilegio mínimo, permisos y propietarios correctos, rutas absolutas, entornos controlados, revisión de tareas/reglas y monitorización de cambios.

Relación con pentesting y certificaciones

Se evalúa priorización y prueba causal, no cantidad de comandos ejecutados.

Caso guiado

Caso A — Básico: configuración consumida por root

pspy muestra un backup root cada minuto; su config es escribible por el grupo del alumno. Observación: consumidor y control coinciden. Experimento: cambiar ruta de salida a marcador y esperar un ciclo. Resultado: archivo root. Conclusión: primitiva de escritura privilegiada confirmada. Siguiendo paso: evaluar impacto mínimo.

Caso de adaptación

Caso B — Hallazgo rojo sin consumidor

linpeas marca /opt/team escribible, pero ninguna tarea, servicio ni binario lo referencia. Experimento: búsqueda de referencias y observación de procesos. Resultado: cero consumo. Conclusión: no es vector actual; no se inventa payload.

Caso C — Transferencia: socket de mantenimiento

Un grupo puede hablar con un socket de daemon root. El alumno debe identificar operaciones permitidas y usar una acción status antes de cualquier cambio. La primitiva es controlar una operación privilegiada, no escribir un fichero.

Caso D — Falso positivo: SUID del propio usuario

Un binario tiene SUID, pero propietario y EUID son el mismo usuario actual. Experimento: comparar UID real/efectivo. Conclusión: no hay ganancia de identidad.

Ejercicios

1. Prioriza cinco hallazgos por consumidor, control y capacidad.
2. Diseña un marcador para una tarea periódica.
3. Explica qué evidencia descarta un directorio escribible.
4. Separa hallazgo, primitiva e impacto en una regla sudo.

Resumen

Enumerar es construir relaciones verificables. La herramienta propone pistas; el alumno demuestra precondiciones, identidad y capacidad.

Chuleta operativa

1. Identidad.
2. Consumidor privilegiado.
3. Superficie controlada.
4. Momento de consumo.
5. Marcador y control.
6. Capacidad exacta.

Referencias

- [Linux credentials\(7\)](#)
- [Concepto interno](#) (enum-privesc-linux)

Navegación

Índice: [curso](#). Siguiendo: [Abuso de sudo](#). Práctica: [cuaderno](#).

Grupos peligrosos (docker, lxd, disk)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Algunos grupos conceden capacidades que equivalen a root aunque tu usuario no lo sea. docker y lxd permiten arrancar un contenedor que monta el sistema de ficheros del host y escribir en el como root. disk da acceso crudo al dispositivo (leer /etc/shadow con debugfs). adm permite leer todos los logs. Es el atajo mas rapido del privesc-modelo.

Estos grupos existen para administrar subsistemas, pero su poder se traduce trivialmente en root: si controlas el demonio de contenedores puedes montar / del host; si lees el disco crudo, saltas todos los permisos de fichero. Por eso id/groups es lo primero que se mira.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Nada mas orientarte con id/groups (privesc-modelo). Si sales en docker/lxd/disk/adm, prioriza este vector sobre todo lo demas.

Cómo identificarla

- id muestra tu usuario en docker, lxd/lxc, disk o adm.
- El binario docker/lxc responde (el demonio esta corriendo).
- Para disk: debugfs puede abrir el dispositivo raiz.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

id para ver tus grupos; si sale docker, prueba docker run -v /:/mnt --rm -it alpine chroot /mnt sh.

Evidencia esperada: Una shell dentro del chroot con acceso de escritura a / del host real.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Objetivo y contexto: Si estas en el grupo docker, arrancas un contenedor que monta TODO el disco del host en /mnt y haces chroot: ya eres root sobre los ficheros reales del host.

Resultado esperado: Una shell root dentro del chroot con acceso a / del host. Desde ahí lees flags, editas /etc/passwd o pones SUID a bash en el host real.

Capa 2: prueba documentada

```
debugfs -w /dev/sda1
```

Objetivo y contexto: Con el grupo disk accedes al dispositivo en crudo, saltando permisos de fichero. debugfs -w permite leer y escribir el sistema de ficheros directamente.

Resultado esperado: Un prompt debugfs. 'cat /etc/shadow' desde ahí ignora permisos. Ajusta /dev/sdaX al dispositivo raíz real (mira lsblk/mount).

Anatomía de los payloads

La primera prueba es `docker run -v /:/mnt --rm -it alpine chroot /mnt sh`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una shell dentro del chroot con acceso de escritura a / del host real.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Mira tus grupos con id.
- docker: arranca un contenedor montando / del host y chroot para escribir como root.
- lxd: importa una imagen y montala con security.privileged para acceder al host.
- disk: usa debugfs sobre el dispositivo raíz para leer /etc/shadow o escribir.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- id no muestra ninguno de esos grupos.

- El demonio correspondiente (docker/lxd) no esta corriendo pese a estar en el grupo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Pertenecer a ciertos grupos es root casi directo: docker/lxd montan el disco del host; disk lo lee crudo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(group-abuse-linux\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Exploits de kernel (ultimo recurso)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un exploit de kernel aprovecha un fallo en el propio núcleo de Linux para ejecutar código como root desde un usuario normal. Casos famosos: DirtyCow, PwnKit (polkit), DirtyPipe. Funcionan cuando la versión del kernel o de un componente del sistema es vulnerable, independientemente de la config.

Se dejan para el FINAL por dos razones: los exploits de kernel pueden causar un kernel panic y tumbar la máquina (arruinando la room), y muchas veces hay un vector de config más limpio y estable. Solo cuando la enumeración (enum-privesc-linux) no dio nada mejor, se recurre a esto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ningún vector de config (sudo, SUID, cron, capabilities) funciona y `uname -r` revela un kernel viejo o hay un componente vulnerable (polkit, sudo Baron Samedit).

Cómo identificarla

- `uname -r` muestra un kernel antiguo con CVE conocido.
- líneas sugiere exploits de kernel aplicables.
- Componentes vulnerables presentes (pkexec para PwnKit, `sudo < 1.9.5p2` para Baron Samedit).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara `uname -a` con CVEs conocidos (DirtyCow, PwnKit, DirtyPipe) para ese rango de versión.

Evidencia esperada: Tras ejecutar el exploit compilado, un prompt root y `uid=0`. Si no aplica, la máquina puede quedar inestable: usalo con cuidado.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
uname -a; cat /etc/os-release
```

Objetivo y contexto: La version exacta del kernel y de la distro es lo que decide que exploit aplica. Un exploit para otra version puede no funcionar o tumbar la maquina.

Resultado esperado: Version del kernel (ej. 4.4.0) y distro. Cruzalo con searchsploit/CVEs; kernels < ~4.8 huelen a DirtyCow, cualquiera con pkexec vulnerable a PwnKit.

Capa 2: prueba documentada

```
searchsploit linux kernel <version>
```

Objetivo y contexto: Busca exploits locales publicos para esa version concreta de kernel. Prioriza los locales (Local Privilege Escalation) sobre los remotos.

Resultado esperado: Titulos con su rango de version. Lee el PoC antes de lanzarlo (exploitar-cve); comprueba que la version encaja para no arriesgar un panic.

Anatomía de los payloads

La primera prueba es `uname -a; cat /etc/os-release`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tras ejecutar el exploit compilado, un prompt root y uid=0. Si no aplica, la maquina puede quedar inestable: usalo con cuidado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Saca la version exacta con `uname -a` y las versiones de sudo/polkit.
- Busca un exploit para esa version y leelo antes de ejecutarlo (exploitar-cve).
- Compila en una maquina compatible (o en la victima si hay gcc) y transfierelo.
- Ejecuta con cuidado; confirma uid=0. Si falla o cuelga, vuelve a los vectores de config.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El kernel/distro esta parcheado contra los CVEs conocidos para esa version.

- Quedan vectores de configuración sin probar (sudo, SUID, cron): prueba esos primero.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si el kernel es viejo, un exploit local eleva a root; deja esto para el final porque puede tumbar la máquina. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(kernel-exploits-linux\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

NFS con no_root_squash

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

no_root_squash es una opción de NFS que hace que el root de tu máquina sea tratado como root en el servidor NFS (en lugar de degradarlo a nobody). Si un share exportado con esa opción está montado en una ruta accesible en la víctima, montas el share desde tu Kali (donde SI eres root), creas ahí un binario SUID root, y lo ejecutas en la víctima para escalar.

Normalmente NFS 'aplasta' (squash) el root remoto para que no pueda actuar como root local. no_root_squash desactiva esa protección: los ficheros que creas como root en tu máquina conservan uid 0 en el servidor. Un binario SUID creado así corre como root al ejecutarlo en la víctima.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves NFS (2049) o /etc/exports revela un share con no_root_squash. showmount lista los exports disponibles.

Cómo identificarla

- showmount -e muestra un export disponible.
- /etc/exports (legible en la víctima) contiene 'no_root_squash'.
- El export está montado o es montable en una ruta útil de la víctima.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Monta el share en tu Kali, crea un binario SUID como root, y ejecútalo desde la víctima por esa misma ruta.

Evidencia esperada: El binario mantiene el bit SUID y euid root al ejecutarse en la víctima.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
showmount -e $IP
```

Objetivo y contexto: Lista los directorios que el servidor NFS exporta y a quien. Es el primer paso: necesitas saber que share montar y si aplica.

Resultado esperado: Rutas exportadas y sus restricciones de host. Un export sin restricción (*) es montable por ti. Luego confirma `no_root_squash` en `/etc/exports`.

Capa 2: prueba documentada

```
sudo mount -t nfs $IP:/export /mnt/nfs && cd /mnt/nfs
```

Objetivo y contexto: Monta el share en tu Kali, donde eres root. Los ficheros SUID que crees aquí como root conservarán uid 0 en la víctima gracias a `no_root_squash`.

Resultado esperado: El share montado en `/mnt/nfs`. Ahora compila un binario que lance bash, hazlo `chown root:root` y `chmod +s`; luego ejecútalo en la víctima desde esa misma ruta.

Anatomía de los payloads

La primera prueba es `showmount -e $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El binario mantiene el bit SUID y euid root al ejecutarse en la víctima.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera los exports con `showmount -e` contra la IP.
- Monta el share en tu Kali (como root).
- Crea ahí un binario que lance una shell y ponle el bit SUID (como root de tu Kali).
- En la víctima, ejecuta ese binario desde la ruta del share: correra como root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `showmount -e` no devuelve ningún export, o el export tiene `root_squash` activo.
- No puedes montar NFS desde tu máquina de ataque (firewall, sin cliente NFS).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un export NFS con no_root_squash te deja crear ficheros SUID como root desde tu maquina y ejecutarlos en la victima. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (nfs-no-root-squash)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Secuestro de PATH

Objetivos de aprendizaje

Demostrar resolución por PATH, entorno heredado y directorio escribible antes de introducir un ejecutable marcador; distinguir cadenas presentes de llamadas alcanzables.

Prerrequisitos

PATH, execvp, shell, permisos de directorio, cron/sudo/SUID y credenciales efectivas.

Fundamentos técnicos

El vector requiere: consumidor privilegiado invoca un nombre sin ruta; usa búsqueda PATH; el PATH efectivo contiene antes un directorio escribible; y la rama se ejecuta. strings solo descubre texto. Exportar PATH en una shell no afecta a un proceso que fija su entorno. En SUID, shells e intérpretes pueden bajar privilegios.

Modelo mental

```
proceso privilegiado -> nombre relativo -> PATH efectivo -> primer ejecutable
                    -> permisos/identidad -> efecto
```

Superficie de ataque

Scripts root, tareas, wrappers, binarios que usan execvp/system, reglas sudo con entorno y servicios que llaman utilidades auxiliares.

Cómo identificarla

- Traza muestra búsqueda o ejecución de nombre relativo.
- PATH del proceso privilegiado se conoce.
- Un directorio anterior es escribible y atravesable.
- Un marcador se ejecuta con identidad esperada.

Preguntas que debo hacerme

1. ¿La llamada usa ruta absoluta?
2. ¿Qué PATH ve el consumidor?
3. ¿Cuál es el primer candidato ejecutable?
4. ¿Puedo escribir y atravesar ese directorio?
5. ¿El intérprete conserva privilegios?

Prueba mínima

Crear un ejecutable marcador con el nombre esperado que escriba id a /tmp, en un directorio de laboratorio ya incluido en el PATH privilegiado. No sustituir binarios reales ni lanzar shell.

Construcción progresiva del payload

1. Confirmar llamada alcanzable.
2. Capturar PATH efectivo.
3. Ordenar candidatos.
4. Verificar permisos de directorio.
5. Introducir marcador.
6. Ejecutar/esperar consumidor, confirmar y limpiar.

Anatomía de los payloads

- Contexto de entrada: cron root ejecuta wrapper que llama backup-helper.
- Sintaxis original: búsqueda por PATH.
- Entrada controlada: archivo /opt/team/bin/backup-helper.
- Transformaciones conocidas: resolución PATH izquierda a derecha.
- Parser final: kernel al ejecutar el candidato.
- Sink: proceso auxiliar como root.
- Primitiva: ejecutar marcador root.
- Payload mínimo: script que guarda id en /tmp/path-proof.
- Significado de cada componente: el nombre coincide con el auxiliar; el marcador registra la identidad sin persistencia.
- Resultado esperado: cron selecciona el candidato anterior al binario legítimo y crea la prueba.
- Control negativo: retirar marcador y observar helper legítimo.
- Restricción observada: exportar PATH del alumno no afecta cron.
- Por qué falla la variante básica: se modificó el entorno equivocado.
- Hipótesis de adaptación: usar un directorio ya presente en PATH de cron.
- Payload adaptado: marcador en /opt/team/bin, si es escribible y precede al legítimo.
- Por qué debería funcionar: modifica un componente que sí pertenece al entorno heredado por el consumidor privilegiado.
- Evidencia: archivo root y traza de la ruta ejecutada.
- Cuándo no funcionaría: PATH fijo sin directorio escribible, ruta absoluta, hash/cache o drop.

Variaciones según el contexto

Shell, execvp y bibliotecas buscan PATH; execve con ruta absoluta no. Sudo puede aplicar secure_path; systemd suele definir entorno explícito; un SUID puede sanearlo o perder EUID al invocar shell.

Filtros y bypasses

Si el PATH privilegiado no es controlable, no existe adaptación mediante export local. Buscar otro consumidor o cerrar la hipótesis.

Evidencias de confirmación

Traza de ruta ejecutada y marcador con identidad privilegiada. Una cadena cat en el binario no confirma llamada.

Escalado de impacto

Demostrar ejecución mínima, restaurar y documentar consumidor/entorno. No reemplazar utilidades del sistema.

Errores frecuentes

- Concluir desde strings.
- Exportar PATH solo en la sesión del alumno.
- Ignorar secure_path.
- Crear /bin/bash como payload y perder EUID.
- No comprobar orden ni permisos x del directorio.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
marcador no se ejecuta	PATH distinto o rama no alcanzada	entorno/traza del consumidor
ejecuta helper legítimo	directorio posterior/cache	orden PATH y type -a/traza
ejecuta marcador como alumno	consumidor no privilegiado	UID/PID correlacionado
sudo ignora PATH	secure_path	sudoers/entorno dentro
shell pierde EUID	modo no privilegiado	marcador directo sin shell y manual Bash

Mitigaciones

Rutas absolutas, PATH mínimo no escribible, entorno fijado, permisos de directorio, evitar shell, drop temprano y revisión de consumidores privilegiados.

Relación con pentesting y certificaciones

Se evalúa la demostración de resolución y entorno, no la creación de un binario llamado cat.

Caso guiado

Caso A — Básico: helper de backup

Cron root tiene PATH /opt/team/bin:/usr/bin; el primer directorio es escribible y el wrapper llama backup-helper. Marcador produce archivo root. Conclusión: PATH hijacking confirmado.

Caso de adaptación

Caso B — PATH exportado no se hereda

Anteponer /tmp en la sesión no cambia el cron. Experimento: capturar PATH del proceso. Resultado: cron lo fija. Conclusión: la variante falla por entorno; si ningún directorio de ese PATH es escribible, se descarta.

Caso C — Transferencia: servicio y execvp

Un servicio root llama compressor mediante execvp y define PATH con un directorio de plugins escribible. El alumno usa marcador, confirma ruta y limpia. Misma primitiva, consumidor distinto.

Caso D — Falso positivo: cadena no alcanzable

strings muestra service, pero la traza del flujo probado nunca ejecuta ese nombre; pertenece a una función de depuración deshabilitada. Conclusión: no hay sink alcanzado.

Ejercicios

1. Ordena candidatos de tres PATH distintos.
2. Diseña un marcador sin shell persistente.
3. Distingue entorno interactivo, sudo y cron.
4. Explica por qué strings es solo una pista.

Resumen

PATH hijacking exige nombre relativo, búsqueda efectiva, directorio escribible anterior y consumidor privilegiado alcanzable.

Chuleta operativa

1. Consumidor/UID.
2. Llamada relativa.
3. PATH efectivo.
4. Orden y permisos.
5. Marcador/control.
6. Drop/limpieza.

Referencias

- [environ\(7\)](#)
- [sudoers\(5\)](#)
- [Concepto interno](#) (path-hijacking)

Navegación

Anterior: [Cron](#). Índice: [curso](#). Práctica: [cuaderno](#).

Python library hijacking

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un script de Python que se ejecuta como root hace 'import modulo', Python busca ese modulo recorriendo una lista de directorios (sys.path) en orden. Si puedes colocar un fichero .py con ese nombre en un directorio que se busque ANTES que el modulo real, o sobrescribir el propio modulo, tu codigo se ejecuta con privilegios de root al importarse. Es el path-hijacking aplicado al sistema de imports de Python.

sys.path incluye, y normalmente en primer lugar, el directorio del propio script; ademas de PYTHONPATH y los site-packages. El programador confía en que se importara el modulo legitimo, pero Python simplemente coge el primero que encuentra. Si el directorio del script es escribible, o el modulo importado lo es, o sudo conserva PYTHONPATH, rompes esa confianza. Misma logica que el privesc-modelo.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando puedes ejecutar un script Python como root (via sudo-abuse) o un cron root lo ejecuta, Y puedes escribir uno de los sitios donde Python busca sus modulos. Es un vector muy comun en rooms de escalada Linux.

Cómo identificarla

- sudo -l te deja ejecutar un script .py como root.
- El directorio del script (o un modulo que importa) es escribible por tu usuario.
- El script hace 'import' de un modulo con nombre poco comun que vive junto a el.
- sudo -l muestra env_keep+=PYTHONPATH.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Revisa sys.path y los permisos de cada directorio anterior al modulo real que importa el script.

Evidencia esperada: Un directorio escribible por ti apareciendo ANTES del modulo legitimo en sys.path.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sudo -l
```

Objetivo y contexto: Punto de partida: ver si puedes ejecutar un script .py como root. Una entrada NOPASSWD sobre un python es el candidato típico de este vector.

Resultado esperado: Líneas '(root) NOPASSWD: /usr/bin/python3 /ruta/script.py'. Abre y lee ese script: te dice que módulos importa y donde vive.

Capa 2: prueba documentada

```
python3 -c 'import sys; print(sys.path)'
```

Objetivo y contexto: Muestra el orden en que Python busca módulos. El primer directorio escribible que preceda al módulo real es tu hueco para inyectar.

Resultado esperado: Lista de directorios. Si el dir del script (o " = dir actual) va primero y es escribible, tienes hijacking directo. Cruza esto con los permisos.

Capa 3: prueba documentada

```
echo 'import os; os.setuid(0); os.system("/bin/bash")' > /dir_escribible/modulo.py
```

Objetivo y contexto: Crea un módulo malicioso con el MISMO nombre que el que importa el script. Al ejecutarse el script como root, importa el tuyo y ejecuta tu código con uid 0.

Resultado esperado: Al lanzar 'sudo python3 /ruta/script.py', obtienes una shell root (id = uid=0). Ajusta el nombre del fichero al módulo exacto que importa el script.

Anatomía de los payloads

La primera prueba es `sudo -l`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un directorio escribible por ti apareciendo ANTES del módulo legítimo en `sys.path`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lee el script root e identifica que modulos importa.
- Mira sys.path y los permisos: ¿puedes escribir el dir del script, el modulo, o un dir que preceda al real?
- Crea un fichero con el nombre del modulo importado que ejecute tu payload (os.setuid(0) + shell).
- Ejecuta el script como root; al importar tu modulo, corre como root. Confirma con id (uid=0).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El script root no hace ningun 'import' de un modulo no estandar.
- Todos los directorios de sys.path anteriores al modulo real son de solo-lectura para ti.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.

3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un script Python que corre como root importa un modulo; si puedes escribir ese modulo (o uno que Python busque antes), tu código corre como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- Concepto fuente de THM Fieldbook (`python-library-hijacking`)
- Referencia técnica externa

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Análisis y abuso de reglas sudo

Objetivos de aprendizaje

Leer usuario destino, comando, argumentos, tags y entorno de una regla sudo; derivar la capacidad real y validar dependencias de versión sin asumir shell root.

Prerrequisitos

UID, argv, entorno, rutas, permisos y documentación de sudoers.

Fundamentos técnicos

Una regla expresa quién, dónde, como quién y qué puede ejecutar. NOPASSWD elimina una autenticación, no las restricciones de comando. Argumentos fijos, NOEXEC, SETENV, secure_path, versión y capacidades del binario cambian el resultado.

Modelo mental

regla -> runas + ruta + argumentos + tags + entorno
-> comportamiento documentado del binario -> capacidad mínima

Superficie de ataque

sudo -l, includes de sudoers, reglas de grupo, sudoedit y entornos preservados. Se analiza la salida completa.

Cómo identificarla

- La invocación exacta coincide con la regla.
- El binario ofrece una función de lectura, escritura o ejecución alcanzable con argumentos permitidos.
- La versión instalada conserva esa función.
- Un marcador confirma la identidad destino.

Preguntas que debo hacerme

1. ¿Runas es root u otro usuario?
2. ¿Argumentos son libres, fijos o comodines?
3. ¿Qué tags y Defaults aplican?
4. ¿Qué entorno llega al proceso?
5. ¿Qué capacidad mínima aporta el binario?

Prueba mínima

Ejecutar la invocación permitida de forma no destructiva y confirmar EUID o acceso al recurso previsto. Consultar manual local y versión antes de recetas externas.

Construcción progresiva del payload

1. Copiar regla exacta.
2. Separar host, runas, tags, ruta y argumentos.
3. Reproducir operación legítima.
4. Identificar función secundaria alcanzable.
5. Usar marcador id o fichero temporal.
6. Comparar con argumento que no coincide.

Anatomía de los payloads

- Contexto de entrada: (backup) NOPASSWD: /usr/bin/cat /srv/app/report.txt.
- Sintaxis original: ruta y argumento exactos.
- Entrada controlada: ninguna; solo ejecución autorizada.
- Transformaciones conocidas: sudo compara ruta, RunAs y argumentos antes de ejecutar.
- Parser final: sudoers y luego cat.
- Sink: lectura como backup.
- Primitiva: leer ese fichero con identidad destino.
- Payload mínimo: invocación exacta.
- Significado de cada componente: sudo -u backup selecciona RunAs; ruta y operando reproducen la regla exacta.
- Resultado esperado: lectura del reporte como backup y rechazo de cualquier operando distinto.
- Control negativo: sustituir ruta por otro fichero.
- Restricción observada: argumento fijo.
- Por qué falla la variante básica: pedir /etc/shadow no coincide.
- Hipótesis de adaptación: estudiar si el fichero permitido es enlace/escribible; no inventar argumento.
- Payload adaptado: solo si otra precondition real lo permite.
- Por qué debería funcionar: la adaptación debe actuar sobre un componente controlable que la regla realmente consuma.
- Evidencia: lectura del reporte y rechazo del control.
- Cuándo no funcionaría: regla distinta, contraseña requerida o política adicional.

Variaciones según el contexto

NOEXEC intenta impedir que ciertos ejecutables lancen otros procesos, con dependencias de plataforma/compilación. SETENV permite mayor control ambiental; secure_path sustituye PATH salvo excepciones documentadas. Debe comprobarse la versión local.

Filtros y bypasses

Una regla con argumentos exactos no se “salta” añadiendo texto al azar. Los comodines de sudoers y del programa son capas distintas. Analiza qué cadena compara sudo y qué parser consume después.

Evidencias de confirmación

Capacidad ejecutada como runas y atribuida a una función permitida. Ver NOPASSWD no confirma escape.

Escalado de impacto

De lectura a escritura o ejecución solo si la regla y el binario lo permiten. GTFOBins sirve como índice, no como sustituto del manual y la versión.

Errores frecuentes

- Leer solo NOPASSWD.
- Ignorar runas no root.
- Cambiar ruta/argumentos y esperar coincidencia.
- Suponer que variables peligrosas sobreviven.
- Ignorar NOEXEC y versión.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
"not allowed to execute"	ruta/argumentos no coinciden	copiar regla y usar <code>sudo -l</code>
función de escape no abre proceso	NOEXEC, versión o build	tags, <code>sudo -V</code> , manual local
PATH manipulado no cambia binario	<code>secure_path</code> o ruta absoluta	<code>sudo -V</code> , entorno dentro del comando
variable desaparece	<code>env_reset/linker</code>	<code>sudo -V</code> y variable inocua
EUID es otro usuario no root	runas específico	<code>id</code> como ese usuario

Mitigaciones

Comandos mínimos con rutas y argumentos exactos, evitar binarios multipropósito, NOEXEC/intercept como defensa adicional, entorno saneado, `secure_path`, logging y revisión periódica.

Relación con pentesting y certificaciones

Se evalúa lectura precisa de la política y capacidad, no buscar cada nombre en GTFOBins.

Caso guiado

Caso A — Básico: lector como usuario backup

La regla permite cat de un reporte como backup. Experimento: invocación exacta y otro fichero como control. Resultado: solo el reporte. Conclusión: capacidad limitada de lectura, no root.

Caso de adaptación

Caso B — Editor con NOEXEC

Una receta conocida intenta lanzar un proceso y falla. Hipótesis: NOEXEC, versión o comando sin función. Experimento: revisar tag, build y una operación interna no ejecutora. Resultado: NOEXEC activo. Conclusión: la receta básica no aplica; se evalúan solo capacidades internas del editor.

Caso C — Transferencia: variable preservada

Una regla permite un script de diagnóstico y conserva una variable de configuración. El alumno demuestra con valor marcador que el script lee esa ruta como usuario destino; después comprueba permisos del archivo. La primitiva es controlar configuración consumida, no "sudo shell".

Caso D — Falso positivo: regla sin contraseña

(root) NOPASSWD: /usr/bin/systemctl status app.service muestra estado, pero argumentos adicionales no coinciden y el pager está deshabilitado. Conclusión: no hay escape demostrado.

Ejercicios

1. Descompón una regla con runas, tags y argumentos.
2. Diseña un control para `secure_path`.
3. Explica diferencia entre `SETENV` y `env_keep`.
4. Evalúa una receta dependiente de versión.

Resumen

Sudo concede una invocación descrita por política. El análisis parte de esa gramática y deriva una capacidad exacta.

Chuleta operativa

1. Runas.
2. Ruta.
3. Argumentos.
4. Tags.
5. Defaults/entorno.
6. Versión y función.
7. Marcador/control.

Referencias

- [sudoers\(5\)](#)
- [Concepto interno](#) (sudo-abuse)

Navegación

Anterior: [Enumeración](#). Siguiente: [SUID](#). Índice: [curso](#).

Sudo con env_keep (LD_PRELOAD)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

LD_PRELOAD fuerza al cargador a inyectar una librería tuya antes que las del sistema en cualquier programa. Si sudo está configurado con env_keep+=LD_PRELOAD, esa variable sobrevive al sudo y puedes precargar una .so maliciosa en un comando que tengas permitido ejecutar con sudo, corriendo tu código como root. Variante avanzada de sudo-abuse.

El admin conserva variables de entorno por comodidad sin darse cuenta de que LD_PRELOAD permite inyectar código. Tu .so define una función (por ejemplo el constructor) que lanza una shell; al cargarla en el proceso privilegiado, esa shell es root.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando sudo -l (sudo-abuse) muestra 'env_keep+=LD_PRELOAD' (o LD_LIBRARY_PATH) y tienes algún comando permitido con sudo, aunque sea inofensivo.

Cómo identificarla

- sudo -l muestra 'env_keep+=LD_PRELOAD' en la política.
- Tienes al menos un comando ejecutable con sudo.
- Puedes compilar en la máquina (gcc presente) o subir la .so ya compilada.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Ejecuta el comando permitido con sudo LD_PRELOAD=/tu.so y observa si tu código corre.

Evidencia esperada: Un prompt de root al ejecutar el comando con la variable precargada.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
gcc -fPIC -shared -o /tmp/pe.so pe.c -nostartfiles
```

Objetivo y contexto: Compila tu librería como shared object. -shared la hace precargable; -nostartfiles evita dependencias que romperían la carga en el proceso víctima.

Resultado esperado: El fichero /tmp/pe.so. Su código (una función _init/constructor que hace setuid(0); system("/bin/bash")) se ejecutará al precargarse.

Capa 2: prueba documentada

```
sudo LD_PRELOAD=/tmp/pe.so <comando_permitido>
```

Objetivo y contexto: Al ejecutar el comando permitido con sudo, LD_PRELOAD (conservado por env_keep) carga tu .so en el proceso root y dispara tu shell.

Resultado esperado: Un prompt de root (id = uid=0). Si da error de carga, revisa que compilaste con -nostartfiles y que env_keep realmente conserva la variable.

Anatomía de los payloads

La primera prueba es gcc -fPIC -shared -o /tmp/pe.so pe.c -nostartfiles. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un prompt de root al ejecutar el comando con la variable precargada.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma env_keep+=LD_PRELOAD en sudo -l.
- Escribe una .so mínima cuyo constructor lance /bin/bash y quite privilegios (setuid(0)).
- Compíllala como shared object.
- Ejecuta un comando permitido con sudo LD_PRELOAD=/ruta/tu.so; obtienes shell root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- sudo -l no muestra env_keep para LD_PRELOAD/LD_LIBRARY_PATH.

- No tienes gcc en la maquina ni forma de subir una .so ya compilada.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si sudo conserva LD_PRELOAD, cargas una librería tuya en un comando permitido y ejecutas código como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(sudo-1d-preload\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Binarios SUID y credenciales efectivas

Objetivos de aprendizaje

Modelar UID real, efectivo y guardado; verificar cuándo SUID se aplica o se ignora; reconocer pérdida de privilegios y derivar capacidades del binario.

Prerrequisitos

Credenciales de proceso, permisos, mounts, execve e intérpretes.

Fundamentos técnicos

Al ejecutar un binario SUID, execve puede establecer el UID efectivo al propietario. No cambia automáticamente el UID real. El efecto se ignora con `no_new_privs`, `mount nosuid` o `tracing`, y Linux ignora SUID en scripts. El programa o un intérprete puede bajar privilegios; Bash sin modo privilegiado restablece EUID cuando difiere del real.

Modelo mental

archivo SUID + propietario -> execve -> RUID/EUID/SUID guardado
-> lógica del binario -> posible drop -> operación sensible

Superficie de ataque

Binarios no estándar, helpers propios, versiones concretas y funciones que leen, escriben o ejecutan con EUID superior.

Cómo identificarla

- Propietario y bit con `find/stat`.
- Mount y `no_new_privs` permiten el efecto.
- `id -ru` e `id -u` difieren dentro del proceso.
- Una función alcanzable usa el EUID antes de bajarlo.

Preguntas que debo hacerme

1. ¿Quién es propietario?
2. ¿Binario ELF o script?
3. ¿Mount `nosuid` o `no_new_privs`?
4. ¿El programa baja privilegios?
5. ¿Qué operación concreta ejecuta con EUID elevado?

Prueba mínima

Enumerar y, en un helper del laboratorio, imprimir RUID/EUID y realizar una operación marcador permitida. No asumir que un prompt `#` refleja identidad.

Construcción progresiva del payload

1. `stat` del candidato.
2. Tipo de fichero y mount.
3. RUID/EUID observados.
4. Flujo de función alcanzable.
5. Marcador de lectura/escritura.
6. Control con `no_new_privs` o copia sin bit.

Anatomía de los payloads

- Contexto de entrada: helper SUID root /opt/lab/report-reader.
- Sintaxis original: acepta un ID de reporte, no una ruta.
- Entrada controlada: ID numérico.
- Transformaciones conocidas: conversión decimal y consulta de allowlist.
- Parser final: lógica del helper.
- Sink: lectura de reporte como EUID 0.
- Primitiva: leer solo reporte permitido.
- Payload mínimo: ID de marcador 7.
- Significado de cada componente: 7 selecciona un reporte permitido sin introducir ruta ni metacarácter.
- Resultado esperado: lectura del marcador con EUID 0 conservada en la traza.
- Control negativo: ID inexistente y ruta textual rechazada.
- Restricción observada: allowlist interna.
- Por qué falla la variante básica: pasar /etc/shadow no es ID.
- Hipótesis de adaptación: ninguna sin otra debilidad.
- Payload adaptado: no procede mientras la allowlist y la conversión sean correctas.
- Por qué debería funcionar: no se afirma una adaptación sin una nueva precondition observable.
- Evidencia: EUID 0 y contenido del marcador.
- Cuándo no funcionaría: nosuid, no_new_privs, propietario no root o drop previo.

Variaciones según el contexto

Una función de lectura puede dar impacto sin shell. Una shell hija puede perder EUID. Capabilities y sudo no son equivalentes a SUID y tienen reglas distintas.

Filtros y bypasses

No se “bypasseea” nosuid con una receta de otro binario. Si el kernel ignora el bit, la precondition falta. La adaptación debe buscar otra capacidad real, no caracteres.

Evidencias de confirmación

EUID superior dentro de una operación sensible y efecto controlado. Listar un bit solo es hallazgo.

Escalado de impacto

Analizar funciones y versión, demostrar capacidad mínima y evitar invocar shells que alteren credenciales sin comprender su modo privilegiado.

Errores frecuentes

- Decir que SUID “siempre” ejecuta como propietario.
- Tratar scripts SUID como binarios.
- Confundir RUID y EUID.
- Usar /bin/bash sin considerar pérdida de privilegios.
- Ignorar mounts y no_new_privs.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
bit existe, EUID no cambia	nosuid/no_new_privs/tracing	mount y /proc/self/status
shell vuelve al usuario	intérprete baja EUID	comparar helper interno y Bash -p documentado
script SUID no eleva	kernel ignora bit en scripts	file y execve(2)
EUID cambia, lectura falla	permisos/MAC/drop previo	momento exacto y logs
propietario no root	identidad distinta	stat e id

Mitigaciones

Eliminar bits innecesarios, reducir superficie, drop temprano e irreversible, rutas/IDs cerrados, mounts nosuid, no_new_privs, revisión de binarios y actualizaciones.

Relación con pentesting y certificaciones

Se evalúa el modelo de credenciales y la función alcanzable, no ejecutar una línea de catálogo.

Caso guiado

Caso A — Básico: lector limitado

Un helper SUID root imprime RUID 1001/EUID 0 y lee un reporte por ID. Conclusión: SUID activo y capacidad de lectura limitada; no implica shell.

Caso de adaptación

Caso B — Copia en /mnt/share

El mismo binario copiado conserva el bit, pero EUID no cambia. Experimento: findmnt -no OPTIONS /mnt/share. Resultado: nosuid. Conclusión: el payload no falla por versión; falta la precondition del mount.

Caso C — Transferencia: helper de impresión

Un binario SUID de otro usuario escribe colas en su directorio. El alumno demuestra escritura con marcador y evalúa si esa identidad aporta acceso útil. La primitiva es cambio de identidad efectiva, no necesariamente root.

Caso D — Falso positivo: script con bit SUID

deploy.sh muestra -rwsr-xr-x, pero se ejecuta con EUID del alumno. Experimento: file, id dentro y manual execve. Conclusión: Linux ignora el bit en scripts.

Ejercicios

1. Interpreta RUID/EUID/SUID guardado en tres procesos.
2. Distingue nosuid de drop de privilegios.
3. Explica por qué una shell hija puede perder EUID.
4. Evalúa un SUID propiedad de usuario backup.

Resumen

SUID modifica credenciales bajo condiciones concretas. La explotación depende de una función sensible ejecutada antes de perder privilegios.

Chuleta operativa

1. Propietario y tipo.
2. Mount.
3. no_new_privs.
4. RUID/EUID.
5. Drop.
6. Función sensible.
7. Marcador/control.

Referencias

- [execve\(2\)](#)
- [GNU Bash Reference Manual](#)
- [Concepto interno](#) (suid)

Navegación

Anterior: [Sudo](#). Siguiente: [Cron](#). Índice: [curso](#).

Inyeccion por comodin (wildcard)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un script root ejecuta algo como 'tar czf backup.tar.gz *' en un directorio, el shell expande el * a la lista de ficheros ANTES de pasarla al comando. Si puedes crear ficheros ahí, creas ficheros cuyo nombre es en realidad una opción del comando (como --checkpoint-action de tar), y el comando ejecuta lo que pongas.

El shell no distingue un fichero llamado '--checkpoint-action=exec=sh script.sh' de una opción legítima: el glob lo expande igual. tar, chown, rsync y otros tienen flags que ejecutan comandos, y así el proceso root corre tu código. Es un privesc-modelo via nombres de fichero.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando cron-abuse o un script root usa un comando con * en un directorio escribible por ti. Muy común con tar en tareas de backup.

Cómo identificarla

- Un cron/script root ejecuta tar/chown/rsync con * en una ruta que puedes escribir.
- El directorio del comodín es de tu propiedad o escribible por tu grupo.
- El comando tiene flags conocidas de ejecución (tar --checkpoint-action).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Crea los ficheros con nombres de flags (--checkpoint=1, --checkpoint-action=...) y espera al siguiente ciclo.

Evidencia esperada: Tras el ciclo, tu payload se ejecuta como root (comprueba con el binario/flag SUID que hayas preparado).

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh
```

Objetivo y contexto: Prepara el payload que el comando privilegiado acabara ejecutando: copia bash con bit SUID para tener luego una shell root.

Resultado esperado: El fichero shell.sh en el directorio del comodín. Aun no hace nada: falta que el comando root lo invoque via las flags.

Capa 2: prueba documentada

```
touch './--checkpoint=1'; touch './--checkpoint-action=exec=sh shell.sh'
```

Objetivo y contexto: Crea ficheros cuyos nombres son opciones de tar. Cuando el cron ejecute 'tar ... *', el glob los mete como argumentos y tar ejecutara shell.sh como root.

Resultado esperado: Tras correr la tarea, /tmp/rootbash existe con bit SUID: lanzalo con -p para shell root. Ajusta las flags al comando concreto (chown/rsync tienen las suyas).

Anatomía de los payloads

La primera prueba es `echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tras el ciclo, tu payload se ejecuta como root (comprueba con el binario/flag SUID que hayas preparado).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica el comando con comodín y el directorio (leyendo el cron/script).
- Crea el payload que ejecutara (p. ej. un .sh que ponga SUID a bash).
- Crea ficheros cuyos NOMBRES sean las flags que hacen que el comando ejecute tu payload.
- Espera a que la tarea corra y recoge tu shell/binario root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El comando no usa comodines o los ficheros se listan explícitamente por nombre.

- El directorio del comodin no es escribible por tu usuario ni grupo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un comando root con * en un directorio donde puedes crear ficheros: nombras ficheros como flags y el shell los expande a argumentos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(wildcard-injection\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Ficheros sensibles escribibles

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ciertos ficheros del sistema definen quien es root y quien puede escalar. Si por una mala configuración de permisos tu usuario puede escribir en `/etc/passwd`, `/etc/shadow` o `/etc/sudoers`, puedes añadir o alterar una cuenta con uid 0 o darte permisos sudo. No hay exploit: es escritura directa a la fuente de la verdad.

Estos ficheros deberían ser solo-lectura para usuarios normales. Cuando alguien afloja los permisos (o un script los deja escribibles), rompes la frontera de confianza mas básica del sistema: `/etc/passwd` todavía permite un hash de password en el segundo campo, así que puedes crear un root con password conocida.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la enumeración (`enum-privesc-linux`) marca `/etc/passwd`, `/etc/shadow` o `/etc/sudoers` como escribibles por ti (`ls -l` o el check de linpeas).

Cómo identificarla

- `ls -l /etc/passwd` muestra permiso de escritura para tu usuario o grupo.
- `/etc/shadow` legible/escribible (linpeas lo resalta).
- Un fichero en `/etc/sudoers.d/` escribible por ti.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

`ls -l /etc/passwd /etc/shadow /etc/sudoers` y busca un permiso `w` para ti.

Evidencia esperada: Un `'w'` en los permisos de alguno de esos ficheros para tu usuario o grupo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ls -l /etc/passwd /etc/shadow /etc/sudoers
```

Objetivo y contexto: Comprueba de un vistazo si tienes escritura en los ficheros que definen identidad y privilegios. Es un check barato con recompensa enorme.

Resultado esperado: Los permisos. Una 'w' para tu usuario/grupo en cualquiera de ellos es escalada directa.

Normalmente solo root escribe; si no, hay mala config.

Capa 2: prueba documentada

```
openssl passwd -1 -salt xyz Passw0rd
```

Objetivo y contexto: Genera un hash valido para /etc/passwd. Si passwd es escribible, anades una linea 'hacker:HASH:0:0:/root:/bin/bash' y creas un root con password conocida.

Resultado esperado: El hash a pegar en el segundo campo. Tras anadir la linea, 'su hacker' con Passw0rd te da uid=0. -1 usa MD5crypt, compatible con /etc/passwd.

Anatomía de los payloads

La primera prueba es `ls -l /etc/passwd /etc/shadow /etc/sudoers`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un 'w' en los permisos de alguno de esos ficheros para tu usuario o grupo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba permisos de /etc/passwd, /etc/shadow y /etc/sudoers(.d).
- Si /etc/passwd es escribible, genera un hash (openssl passwd) y anade una linea de usuario con uid 0.
- Cambia a ese usuario con su y la password que pusiste.
- Confirma uid=0. Alternativa: darte una linea NOPASSWD en sudoers.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- /etc/passwd, /etc/shadow y /etc/sudoers(.d) son todos solo-lectura para tu usuario/grupo.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si puedes escribir `/etc/passwd`, `/etc/shadow` o `/etc/sudoers`, te conviertes en root editando una línea. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (writable-sensitive-files)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.